mjy wo processes chahy jo hamara ubuntu use chala raha hy.
↳ ps aux | grep ubuntu

Date: 14/7/26

Day-5 (Pro linux commands)

→ AWK
→ SED
→ GREP

grep, sed, and awk are powerful linux text processing commands.

Devops engineers use them to:

- Search application and system logs
- find errors and warnings
- Extract useful information
- modify configuration files
- process command output
- Analyze CSV and text files
- Automate deployment scripts
- filter monitoring data

Simple memory Trick:

grep → Search
Sed → Edit
awk → Extract and process columns

1. grep: → Global Regular expression (pattern)

purpose:

grep searches for specific words, patterns, or regular expressions inside files or command output.

Basic Syntax:

grep [options] "pattern" filename

Simple example:

grep "error" application.log

Meaning: finds all lines containing the word 'error' in application.log.

CamScanner

(2)

Date: 14/7/26

Important grep options:

-i :

purpose: Ignores uppercase and lowercase differences.

grep -i "error" application.log

This maches:

error
Error
ERROR

-n :

purpose: Shows line numbers with matches lines.

grep -n "error" application.log

Example output:

15 : Database connection error

Here, 15 is the line number.

-v : $

purpose: Shows lines that do not match the pattern.

grep -v "success" application.log

This displays all lines except those containing success.

-c :

purpose: Counts the number of matching lines.

grep -c "error" application.log

-w:

purpose: matches a complete word only.

grep -w "user" file.text

it matches user, but not username.

CamScanner

Date: 14/7/26

-r or -R:

purpose: Searches recursively inside directories and subdirectories.

grep -r "database" /etc/

-l:

purpose: Shows only filenames containing a match.

grep -l "error" *.log

-L:

purpose: Shows filename that don't contain the pattern

grep -L "error" *.log

-A:

purpose: Shows lines after the matching line

grep -A 3 "error" application.log

Shows the matching line and the next three lines.

-B:

purpose: Shows lines before the matching line

grep -B 3 "error" application.log

-C:

purpose: Shows lines before and after the match

grep -C 2 "error" application.log

-E:

purpose: Enables extended regular expressions.

grep -E "error|warning" application.log

This finds lines containing either error or warning.

The command egrep is an older equivalent of:

grep -E

CamScanner

Date: 14/7/26

-F:

purpose: Searches for fixed text instead of a regular expression.

grep -F "192.168.1.10" access.log

The older equivalent is:

fgrep

--color = auto:

purpose: Highlights the matching text.

grep --color=auto "error" application.log

Regular expressions with grep:

Regular expressions are patterns used for advanced searching.

^:

purpose: Matches the beginning of a line.

grep "^ERROR" application.log

finds lines starting with ERROR.

$:

purpose: matches the end of a line.

grep "failed$" application.log

finds lines ending with failed.

.:

purpose: Matches any single character.

grep "s.rver" file.text

It may match:

Server

Sarver

S1rver

MIGHTY PAPER PRODUCT

CamScanner

Date: 14/7/26

*

*:

purpose: matches zero or more occurences of the previous character or pattern.

grep "go*d" file.txt

It can match:

gd
god
good
goood

[ ]:

purpose: matches one character from a group.

grep "[Ee]rror" application.log

matches both:

Error
error

[^]:

purpose: matches characters not included in the group.

grep "[^0-9]" file.txt

finds lines containing non-numeric characters.

[0-9]

purpose: matches any digits from 0 to 9.

grep "[0-9]" file.txt

|:

purpose: means OR when used with grep -E.

grep -E "error|failed|warning" application.log

CamScanner

Date: 14/7/26

Multiple patterns:

Using -e:

grep -e "error" -e "warning" application.log

Using extended regular expressions:

grep -E "error|warning" application.log

Searching command output:

grep is frequently used with pipes.

ps aux | grep nginx

Meaning: Searches the process list for nginx

systemctl list-units | grep running

Meaning: shows running systemd units.

docker ps | grep nginx

Meaning: Searches running containers for Nginx.

Devops uses of grep:

Devops engineers use grep to:

1. find errors in log files.
2. Search configuration files
3. check running processes
4. filter Docker and Kubernetes output
5. Search deployment logs
6. find IP addresses and HTTP status codes
7. Locate environment variables
8. check whether services are running.

CamScanner

sed -n '/INFO/p' app.log
sed line by line kam krta.

Date: 20/7/26

2. Sed: → Stream Editor:

Purpose: Sed searches, replaces, deletes, inserts, and modifies text without opening the file in a text editor.

Basic Syntax:

sed [options] 'command' filename

How Sed works:

By default, sed:

1. Reads the input line by line
2. Applies the specified operation
3. Prints the result on the terminal
4. Does not modify the original file unless -i is used.

Important Sed operations:

1. Substitute Text: s

The most common sed operation is substitution.

sed 's/old/new/' file.txt

Meaning: Replaces the first occurence of old with new on each line.

Substitute All Occurrences: g

sed 's/old/new/g' file.txt

Meaning: Replaces every occurrence of old with new.
'g' means global replacement.

Ignore Case: I

sed 's/error/issue/gI' application.log

This replaces:

error
Error
ERROR

CamScanner

if you want change from line 1 to 10
sed '1,10 s/INFO/log/g' app.log

Date: 20/7/26

Modify the original file: -i

Sed -i 's/old/new/g' file.txt

Meaning: changes the original file directly

Important Warning:

Always create a backup before editing an important configuration file.

Sed -i.bak 's/old/new/g' file.txt

This creates:

file.txt

file.txt.bak

Print specific lines: -n and p

Sed -n '5p' file.txt

Meaning: Prints lines 5 only

Sed -n '5,10p' file.txt

Meaning: Prints line 5 through 10.

Without -n, selected lines may appear twice because Sed prints normally and p prints again.

Delete lines: d

Sed '5d' file.txt

Meaning: Deletes line 5 from the displayed output.

Sed '5,10d' file.txt

Meaning: Deletes lines 5 through 10.

Sed '/debug/d' application.log

Meaning: Removes lines containing debug from the output

To modify the file:

Sed -i '/debug/d' application.log

MIGHTY PAPER PRODUCT

CamScanner

→ sed '1,10 s/INFO/log/g; 1,10p; 11q' app.log
it change from line 1 till 10 & print that line as well then quit.

Date: 20/7/26

Insert Text Before a line: i

sed '3i\New line added' file.txt

Meaning: Inserts text before line 3

Append Text after a line: a

sed '3a\New Line added' file.txt

Meaning: ~~inserts text before~~ add text after line 3

change an Entire Line: c

sed '3c\This is the replacement line' file.txt

Meaning: Replaces line 3 completely.

Use a pattern as an address:

sed '/ERROR/p' application.log

This prints matching lines, but because normal printing is still enabled, they may appear twice.

Correct form:

sed -n '/ERROR/p' application.log

Replace Text on specific lines:

Replace text only on line 5:

sed '5s/old/new/' file.txt

Replace text from line 5 to 10:

sed '5,10s/old/new/g' file.txt

Delete Blanks lines:

sed '/^$/d' file.txt

Meaning: Removes empty lines.

Removes leading spaces:

sed 's/^[[:space:]]*//' file.txt

CamScanner

Date: 20/7/26

Remove Trailing spaces:

```
Sed 's/[[:space:]]*$//' file.txt
```

Remove comments:

```
Sed '/^#/d' config.conf
```

Meaning: Removes lines beginning with #

To remove comments and blanks lines:

```
Sed '/^#/d; /^$/d' config.conf
```

Use Another Delimiter:

Normally, / separates the search and replacement strings:

```
Sed 's/old/new/g'
```

for paths and URLs, another delimiter is easier:

```
Sed 's|/var/www/old|/var/www/new|g' file.txt
```

This is easier to read than escaping every slash.

Extended Regular Expressions:-E

```
Sed -E 's/error|warning/issue/g' file.txt
```

purpose: Enables extended regular expressions.

Capturing Groups:

```
Sed -E 's/(user)=([^ ]+)/\1=hidden/' file.txt
```

Here:

\1 = first captured group

\2 = second captured group

MIGHTY PAPER PRODUCT

CamScanner

Date: 20/7/26

DevOps uses of Sed:

Devops engineers use sed to:

1. change ports in configuration files.
2. updates server names
3. Replace environment values
4. Delete comments and blanks lines
5. modify deployment files
6. update version numbers
7. change URLs and paths
8. Edit configuration files automatically in shell scripts

Important Sed Safety Rule:

Before changing a production configuration file, use:

```
Sed 's/old/new/g' config.conf
```

check the output first.

Then use:

```
Sed -i.bak 's/old/new/g' config.conf
```

This saves a backup.

MIGHTY PAPER PRODUCT

CamScanner

if you don't want to do shell scripting then do commands with awk.

CSV comma separated values

(12)

Date: 20/7/26

3. awk → awk is named after its creators:

full form:

"Aho, Weinberger, and Kernighan"

In awk we need formatted data.

purpose:

awk processes text line by line and is mainly used to extract columns, filter records, calculate values, and create reports.

Basic Syntax:

awk 'pattern { action }' filename

Another common form is:

Command | awk '{ action }' filename

How awk Reads Data:

awk divides every line into fields or columns.

Suppose a file contains:

Ali devops 50000
Sara developer 60000
ahmed admin 45000

The fields are:

$1 = first column
$2 = second column
$3 = third column
$4 = complete line

prints complete lines:

awk '{print $0}' employees.txt

purpose: prints every complete line.

Prints the first column:

awk '{print $1}' employees.txt

MIGHTY PAPER PRODUCT

CamScanner

awk records পড়ে যখন তা structured ho,

Date: 20/7/26

Prints multiple columns:

```
awk '{print $1, $3}' employees.txt
```

Meaning: Prints columns 1 and 3.

Important Built-in Variables:

$0:

purpose: Represents the complete current line.

```
awk '{print $0}' file.txt
```

$1, $2, $3:

purpose: Represents individual fields.

```
awk '{print $1, $2}' file.txt
```

NR:

full form: Number of Record

Purpose: Shows the current line number

```
awk '{print NR, $0}' file.txt
```

NF:

full form: Number of fields

purpose: Shows the total number of columns in the current line

```
awk '{print NF}' file.txt
```

$NF:

purpose: Represents the last column.

```
awk '{print $NF}' file.txt
```


CamScanner

Date: 20/7/26

$(NF-1):

purpose: Represents the second last column.

```
awk '{print $(NF-1)}' file.txt
```

FS:

full form: field separator

purpose: Defines the input separator

```
awk 'BEGIN {FS=":"} {print $1}' /etc/passwd
```

OFS:

full form: output field separator

purpose: Defines the separator used in output

```
awk 'BEGIN {OFS="|"} {print $1, $2}' file.txt
```

FILENAME:

Purpose: Shows the current filename.

```
awk '{print FILENAME, $0}' file.txt
```

Field Separators:

By default, awk separates fields using spaces or tabs.

Colon-Separated Data:

```
awk -F: '{print $1}' /etc/passwd
```

-F: means that : is the field separator.

Comma-Separated Data:

```
awk -F, '{print $1, $3}' employees.csv
```

MIGHTY PAPER PRODUCT

CamScanner

Date: 20/7/26

Pipe-Separated Data:

```
awk -F'|' '{print $1, $2}' file.txt
```

Pattern filtering:

100 500
8330
2000
13,830

Match a word:

```
awk '/error/ {print $0}' application.log
```

Meaning: Prints lines containing error.

Ignore case:

```
awk 'tolower($0) ~ /error/ {print}' application.log
```

Numeric Conditions:

```
awk '$3 > 50000 {print $1, $3}' employees.txt
```

Meaning: Prints employees whose third-column value is greater than 50,000.

String Conditions:

```
awk '$2 == "devops" {print $1}' employees.txt
```

Meaning: Prints names where the second column is exactly devops.

Not Equal:

```
awk '$2 != "admin" {print $0}' employees.txt
```

Multiple conditions:

AND condition:

```
awk '$2 == "devops" && $3 > 40000 {print $1}' employees.txt
```

OR Condition:

```
awk '$2 == "devops" || $2 == "admin" {print $1}' employees.txt
```


CamScanner

Date: 20/7/26

BEGIN Block:

purpose: BEGIN runs before awk reads the file.

awk 'BEGIN {print "DevOps Reports"}{print $0}' file.txt

Commonly used to set Separators:

awk 'BEGIN {FS=":"}{print $1}' /etc/passwd

END Block:

purpose:

End runs after all lines have been processed.

awk 'END {print "Total lines:", NR}' file.txt

Calculations with awk:

Calculate Total:

awk '{Sum += $3} END {print "Total:", Sum}' employees.txt

Calculate Average:

awk '{Sum += $3} END {print "Average:", Sum/NR}' employees.txt

Count Matching Lines:

awk '/error/ {count++} END {print "Errors:", count}' application.log

Find Maximum Value:

awk 'NR==1 || $3 > max {max = $3} END {print max}' employees.txt

Find Minimum Value:

awk 'NR==1 || $3 < min {min = $3} END{print min}' employees.txt

MIGHTY PAPER PRODUCT

CamScanner

Date: 20/Feb

Formatting Output with printf:

purpose:
printf gives better control over output formatting.

```
awk '{printf "User: %-10s Salary: %6\n", $1, $3}' employees.txt
```

Common format symbols:

%s = string
%d = integer
%f = decimal number
\n = new line
\t = tab

DevOps Uses of awk:

DevOps engineers use awk to:

1. Extract CPU and memory information
2. Process access logs
3. Analyze HTTP status codes
4. Extract usernames from /etc/passwd
5. Process CSV files
6. Calculate totals and average
7. Extract columns from command output
8. Generate monitoring reports
9. find disk usage values
10. process Docker and Kubernetes output.

Difference Between grep, sed, and awk

Command	Main purpose	Simple meaning
grep	Search lines	find text
sed	Edit text	Replace or delete text
awk	process columns	Extract, filter, and calculate

CamScanner

Date: 20/7/26

When to use which Command:

* Use grep when:
  You need to find lines containing a word.
  grep "ERROR" application.log

* Use Sed when:
  you need to replace or delete text.
  Sed 's/localhost/production-server/g' config.txt

* Use awk when:
  you need to work with columns.
  awk '{print $1, $5}' access.log

Commands used [redacted] Together:

grep with awk:
grep "ERROR" application.log | awk '{print $1, $2, $5}'
Meaning: finds error lines and extracts selected columns.

grep with Sed:
grep "ERROR" application.log | Sed 's/ERROR/ISSUE/g'
Meaning: finds error lines and changes the displayed word ERROR to ISSUE.

Sed with awk:
Sed '/^#/d' config.txt | awk '{print $1}'
Meaning: Removes comment lines and prints the first column.

MIGHTY PAPER PRODUCT

CamScanner

Date: 20/7/26

All Three Together:

```
grep "ERROR" application.log |
sed 's/ERROR/critical/' |
awk '{print $1, $2, $0}'
```

Meaning:

grep = finds error lines
sed = replaces ERROR with critical
awk = format the final output

DevOps Example Areas:

Log Analysis:

grep = find errors
sed = clean or replace text
awk = extract date, IP, status code, or message.

Configuration Management:

grep = check whether a setting exists
sed = change the setting
awk = extract configuration values

Monitoring:

grep = filter important output
sed = clean output
awk = calculate and format values

CI/CD Pipelines:

grep = detect success or failure
sed = update versions and variables
awk = produce reports

CamScanner

Date: 20/7/26

Important Notes for DevOps Engineers:

1. Test sed changes before using sed -i
2. Creates a backup before editing production configuration files.
3. Use quotes around patterns containing spaces
4. Use single quotes for most grep, sed, and awk expressions.
5. Remember that linux commands are case-sensitive.
6. Test commands on sample files before using production logs
7. Use pipes to combine these tools with other linux commands.
8. Avoid modifying active production files without backup and validation.

final Memory Notes:

grep → which lines do I need?
Sed → what text should I change?
awk → which columns and calculations do I need?

Notebook Summary:

grep Searches lines
sed edits text streams
awk processes records and fields

grep finds data
sed changes data
awk extracts, filters, formats, and calculates data.

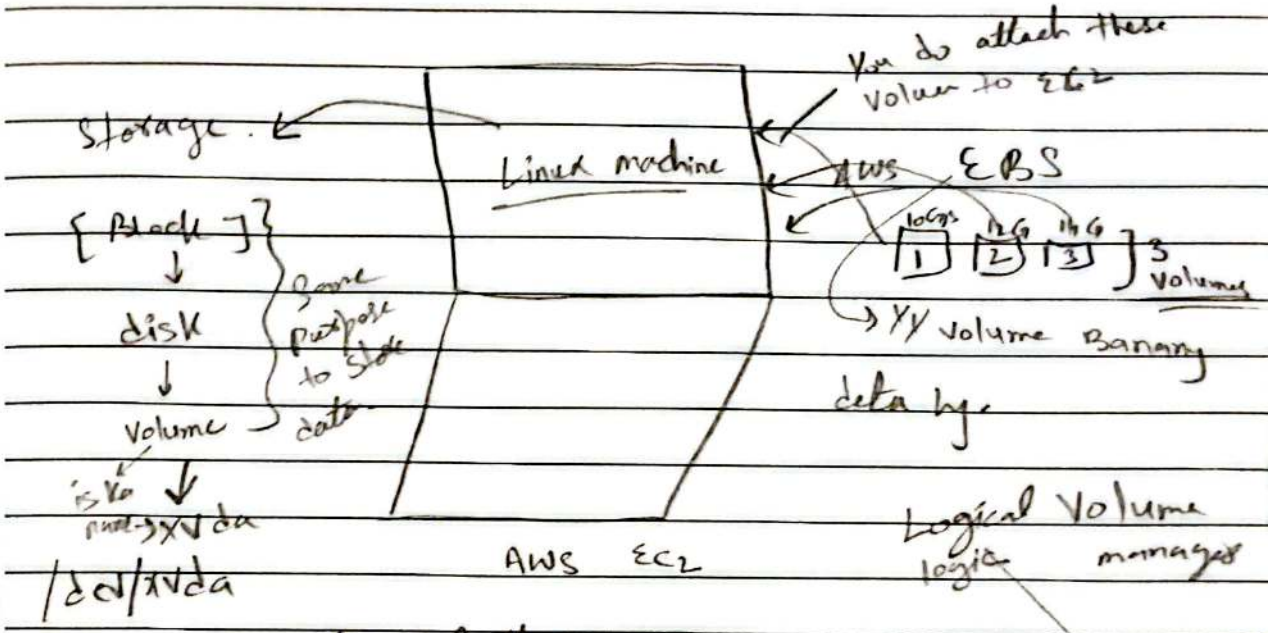
CamScanner

Amazon Elastic Block store

Date: 21/7/24

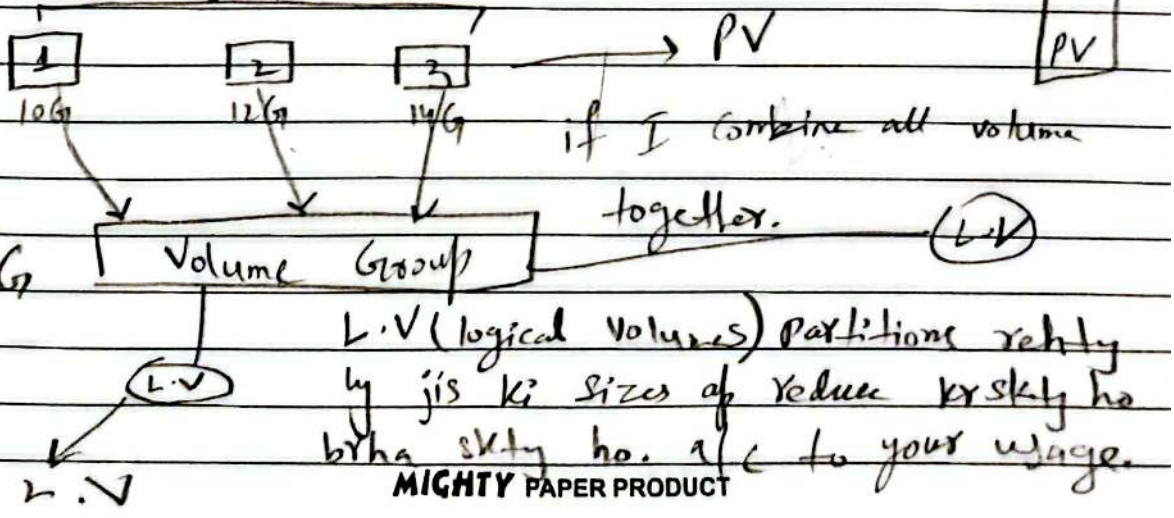
Linux Volume management:

- Introduction to linux volumes and AWS EBS
- Physical vs logical volumes vs Volume Groups in linux
- Mounting volumes in linux
- Managing AWS EBS on EC2 instances
- Introduction to LVM (logical volume manager)
- Using LVM with EBS for Dynamic Storage management (Notes).



By default AWS give 8GB I Hy so If wo Ram par jai to xy volume extra b add kr skty hy.

1. lsblk → sary blocks dikhai gta.
2. df -h → Mount point or storage dikhata.



MIGHTY PAPER PRODUCT

CamScanner

(22) Sudo su → root user ban jaeo

Date: 21/7/26

L.V → lets suppose total 36G hy to mjy
agr 10G os my sy chaiy to hy
skta hun.

Perform Practical LVM

first sudo su

then lvm

now create Physical volume

PV create /dev/nvme0n1 /dev/nvme2n1 /dev/nvme3n1

↳ Now created successfully.

To see → PVS

Now create volume group:

Vgcreate tws_vg /dev/nvme1n1 /dev/nvme2n1

Now create logical Vdume (lv): (logical volume)

lvcreate -L 10G -n tws_lv tws_vg

Now → PVdisplay → To see PV all information.

→ lvdisplay

Date: 21/7/26

Now for mounting:

need to create first directory.

→ mkdir /mnt/tws_lv_mount

Now abi jo bana os formed krdo.

→ mkfs.ext4 /dev/tws_vg/tws_lv

Now mount → jaha mount krna

mount /dev/tws_vg/tws_lv /mnt/tws_lv_mount

Now lsblk, df -h

↓ is my to bina mount k show hota hy volume

↘ is sy after mount show hota hy.

iss volume management sy my memory attach kr skta hy. Is sy memory khatam hi ne ho ghi

attach:

sy disk my block add kr dety ho

mount:

sy ap os block ko usable bana dety ho.

umount /mnt/tws_lv_mount

Disk k leye

mkfs -t ext4 /dev/nvme3n1

now again → mount /dev/nvme3n1 /mnt/tws_disk_mount

MIGHTY PAPER PRODUCT

CamScanner

Date:

We can extend logical volumes

→ lvm →

lvextend -L +5G /dev/aws_vg/aws_lv

End - day - 5

MIGHTY PAPER PRODUCT

CamScanner